

TRABAJO 1. ÍNDICES Y RESTRICCIONES EN ORACLE.

JOGER GABRIEL CASTILLO QUITIAN

RAY SEBASTIAN DE JESÚS GONZÁLEZ PUELLO

FUNDACIÓN UNIVERSITARIA INTERNACIONAL DE LA RIOJA

BASES DE DATOS AVANZADAS

DEIVIS EDUARD RAMIREZ MARTINEZ

04 DE JUNIO DE 2026

COLOMBIA

TABLA DE CONTENIDO

INTRODUCCIÓN.....	3
CONSULTA DE ÍNDICES EXISTENTES	3
Desactivación de restricciones.....	4
Inserción de tuplas que violan restricciones	5
RE-ACTIVACIÓN DE RESTRICCIONES	6
CREACIÓN DE LA TABLA DEPARTMENTS₂.....	7
INSERCIÓN DE TRES TUPLAS EN DEPARTMENTS₂.....	8
persistencia de datos al cerrar sesión.....	8
BLOQUE ANÓNIMO CON TRANSACCIÓN.....	9
DESHACER UNA TRANSACCIÓN CON ROLLBACK	9
ARCHIVOS REDO EN ORACLE.....	10
repositorio de github.....	11
CONCLUSIONES.....	11

INTRODUCCIÓN

En este trabajo se exploraron los mecanismos de índices y restricciones en Oracle 21c utilizando el esquema HR. El objetivo fue comprender cómo se gestionan las restricciones de integridad, cómo se pueden desactivar temporalmente para permitir inserciones anómalas, y cómo funciona el control de transacciones mediante COMMIT y ROLLBACK. También se abordó el concepto de archivos Redo y su importancia en la recuperación de la base de datos.

CONSULTA DE ÍNDICES EXISTENTES

En Oracle, los índices asociados a PRIMARY KEY y UNIQUE constraints son creados automáticamente. Para listar los índices disponibles en EMPLOYEES y DEPARTMENTS, se consultaron las constraints de tipo P (Primary Key) y U (Unique).

Consulta ejecutada:

```
SELECT constraint_name AS index_name,
       table_name,
       CASE constraint_type
         WHEN 'P' THEN 'PRIMARY KEY'
         WHEN 'U' THEN 'UNIQUE'
       END AS index_type,
       status
FROM user_constraints
WHERE table_name IN ('EMPLOYEES', 'DEPARTMENTS')
AND constraint_type IN ('P', 'U')
ORDER BY table_name, constraint_name;
```

Ilustración 1. Script 1.

Resultado obtenido:

Hoja de Trabajo

Generador de Consultas

```
-- =====
-- 1. Consultar indices disponibles (via constraints PK/UK)
-- =====

SELECT constraint_name AS index_name,
       table_name,
       CASE constraint_type
         WHEN 'P' THEN 'PRIMARY KEY'
         WHEN 'U' THEN 'UNIQUE'
       END AS index_type,
       status
FROM user_constraints
WHERE table_name IN ('EMPLOYEES', 'DEPARTMENTS')
AND constraint_type IN ('P', 'U')
ORDER BY table_name, constraint_name;
```

Resultado de la Consulta x

    SQL | Todas las Filas Recuperadas: 2 en 0,676 segundos

	INDEX_NAME	TABLE_NAME	INDEX_TYPE	STATUS
1	SYS_C008234	DEPARTMENTS	PRIMARY KEY	DISABLED
2	SYS_C008236	EMPLOYEES	PRIMARY KEY	DISABLED

Ilustración 2. Índices en tablas con nombres "employees" y "departments"

En EMPLOYEES se identificó la PRIMARY KEY sobre EMPLOYEE_ID y la UNIQUE sobre EMAIL. En DEPARTMENTS se identificó la PRIMARY KEY sobre DEPARTMENT_ID. Todos los índices se encontraban activos (ENABLED).

DESACTIVACIÓN DE RESTRICCIONES

Para permitir inserciones que normalmente violarían las reglas de integridad, fue necesario desactivar las restricciones de ambas tablas. Se utilizó un bloque PL/SQL que desactiva primero las FOREIGN KEY dependientes, luego las PRIMARY KEY y UNIQUE, garantizando el orden correcto.

Código ejecutado:

```
BEGIN
-- Desactivar FOREIGN KEY que dependen de EMPLOYEES
FOR c IN (SELECT constraint_name, table_name
          FROM user_constraints
          WHERE constraint_type = 'R'
          AND table_name IN ('DEPARTMENTS', 'JOB_HISTORY'))
LOOP
    EXECUTE IMMEDIATE 'ALTER TABLE ' || c.table_name ||
        ' DISABLE CONSTRAINT ' || c.constraint_name;
END LOOP;

-- Desactivar PRIMARY KEY y UNIQUE de EMPLOYEES
FOR c IN (SELECT constraint_name, table_name
          FROM user_constraints
          WHERE table_name = 'EMPLOYEES'
          AND constraint_type IN ('P', 'U'))
LOOP
    EXECUTE IMMEDIATE 'ALTER TABLE ' || c.table_name ||
        ' DISABLE CONSTRAINT ' || c.constraint_name;
END LOOP;

-- Desactivar resto de restricciones de DEPARTMENTS
FOR c IN (SELECT constraint_name, table_name
          FROM user_constraints
          WHERE table_name = 'DEPARTMENTS')
LOOP
    EXECUTE IMMEDIATE 'ALTER TABLE ' || c.table_name ||
        ' DISABLE CONSTRAINT ' || c.constraint_name;
END LOOP;

DBMS_OUTPUT.PUT_LINE('Todas las restricciones desactivadas');
END;
```

Ilustración 3. Script 2.

Verificación de desactivación:

```
-- Verificar desactivación
SELECT constraint_name, table_name, status
FROM user_constraints
WHERE table_name IN ('EMPLOYEES', 'DEPARTMENTS')
ORDER BY table_name;
```

Ilustración 4. Verificación de la desactivación.

Antes de la desactivación:

SQL | Todas las Filas Recuperadas: 11 en 0,027 segundos

	CONSTRAINT_NAME	TABLE_NAME	CONSTRAINT_TYPE	STATUS
1	SYS_C008221	DEPARTMENTS	C	DISABLED
2	SYS_C008234	DEPARTMENTS	P	DISABLED
3	SYS_C008240	DEPARTMENTS	R	DISABLED
4	SYS_C008223	EMPLOYEES	C	ENABLED
5	SYS_C008224	EMPLOYEES	C	ENABLED
6	SYS_C008225	EMPLOYEES	C	ENABLED
7	SYS_C008226	EMPLOYEES	C	ENABLED
8	SYS_C008236	EMPLOYEES	P	DISABLED
9	SYS_C008241	EMPLOYEES	R	DISABLED
10	SYS_C008242	EMPLOYEES	R	DISABLED
11	SYS_C008243	EMPLOYEES	R	DISABLED

Ilustración 5. Antes de la desactivación.

Después de la desactivación:

	CONSTRAINT_NAME	TABLE_NAME	STATUS
1	SYS_C008221	DEPARTMENTS	DISABLED
2	SYS_C008234	DEPARTMENTS	DISABLED
3	SYS_C008240	DEPARTMENTS	DISABLED
4	SYS_C008226	EMPLOYEES	ENABLED
5	SYS_C008236	EMPLOYEES	DISABLED
6	SYS_C008241	EMPLOYEES	DISABLED
7	SYS_C008242	EMPLOYEES	DISABLED
8	SYS_C008243	EMPLOYEES	DISABLED
9	SYS_C008224	EMPLOYEES	ENABLED
10	SYS_C008223	EMPLOYEES	ENABLED
11	SYS_C008225	EMPLOYEES	ENABLED

Ilustración 6. Después de la desactivación.

INSERCIÓN DE TUPLAS QUE VIOLAN RESTRICCIONES

Con las restricciones desactivadas, se insertaron tres registros problemáticos:

a) **Empleado sin department_id (viola FOREIGN KEY):**

```
-- Empleado sin department_id (viola FK)
INSERT INTO employees (employee_id, first_name, last_name, email, hire_date, job_id)
VALUES (999, 'Prueba', 'SinDepto', 'prueba1@x.com', SYSDATE, 'SA_REP');
```

Ilustración 7. Error por Foreign key.

b) **Empleado con email duplicado (viola UNIQUE):**

```
-- Empleado con email duplicado (viola UNIQUE)
INSERT INTO employees (employee_id, first_name, last_name, email, hire_date, job_id)
VALUES (998, 'Prueba', 'EmailDup', 'SKING', SYSDATE, 'SA_REP');
```

Ilustración 8. Error por Unique.

c) **Departamento de prueba:**

```
-- Departamento de prueba
INSERT INTO departments (department_id, department_name, location_id)
VALUES (999, 'Depto Prueba', 1700);
```

Ilustración 9. Departamento de prueba.

Todas las inserciones fueron exitosas debido a que las restricciones estaban desactivadas.

```
1 fila insertadas.

1 fila insertadas.

1 fila insertadas.
```

Ilustración 10. Inserciones exitosas.

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL
1	999	Prueba	SinDepto	pruebal@x.com
2	998	Prueba	EmailDup	SKING

Ilustración 11. Resultado con duplicados y sin departamento.

RE-ACTIVACIÓN DE RESTRICCIONES

Al intentar reactivar las restricciones, se observó el siguiente comportamiento:

- La restricción UNIQUE del email (SYS_008225 o similar) **falló** debido al email duplicado.
- La restricción FOREIGN KEY de department_id (SYS_C008241) **falló** debido al empleado 999 sin departamento.
- Las restricciones PRIMARY KEY y las de DEPARTMENTS se activaron correctamente.

```
FALLÓ: SYS_C008236 - ORA-02437: no se puede validar (HR.SYS_C008236) - clave primaria violada
Activada: SYS_C008240
FALLÓ: SYS_C008244 - ORA-02270: no hay ninguna clave única o primaria correspondiente para esta lista de columnas
Activada: SYS_C008245
FALLÓ: SYS_C008246 - ORA-02270: no hay ninguna clave única o primaria correspondiente para esta lista de columnas
Activada: SYS_C008240
Activada: SYS_C008221
FALLÓ: SYS_C008234 - ORA-02437: no se puede validar (HR.SYS_C008234) - clave primaria violada

Procedimiento PL/SQL terminado correctamente.
```

Ilustración 12. Re-activación de restricciones.

Verificación del estado final:

```
-- Verificar estado final
SELECT constraint_name, table_name, status
FROM user_constraints
WHERE table_name IN ('EMPLOYEES', 'DEPARTMENTS')
ORDER BY table_name;
```

Ilustración 13. Verificación del estado final.


```
-- Verificar estado final
SELECT constraint_name, table_name, status
FROM user_constraints
WHERE table_name IN ('EMPLOYEES', 'DEPARTMENTS')
ORDER BY table_name;
```

Salida de Script x Resultado de la Consulta x

Todas las Filas Recuperadas: 11 en 0,023 segundos

	CONSTRAINT_NAME	TABLE_NAME	STATUS
1	SYS_C008221	DEPARTMENTS	ENABLED
2	SYS_C008234	DEPARTMENTS	DISABLED
3	SYS_C008240	DEPARTMENTS	ENABLED
4	SYS_C008226	EMPLOYEES	ENABLED
5	SYS_C008236	EMPLOYEES	DISABLED
6	SYS_C008241	EMPLOYEES	DISABLED
7	SYS_C008242	EMPLOYEES	DISABLED
8	SYS_C008243	EMPLOYEES	DISABLED
9	SYS_C008224	EMPLOYEES	ENABLED
10	SYS_C008223	EMPLOYEES	ENABLED
11	SYS_C008225	EMPLOYEES	ENABLED

Ilustración 14. Resultado por verificación estado final.

Esto demuestra que Oracle no activa una restricción si los datos existentes la violan.

CREACIÓN DE LA TABLA DEPARTMENTS2

Se creó una copia vacía de DEPARTMENTS:

```
-- =====
-- 6. Crear DEPARTMENTS2
-- =====
CREATE TABLE departments2 AS SELECT * FROM departments WHERE 1=0;
SELECT COUNT(*) FROM departments2;
```

Ilustración 15. Script para crear departments2.

```
-- =====
-- 6. Crear DEPARTMENTS2
-- =====
CREATE TABLE departments2 AS SELECT * FROM departments WHERE 1=0;
SELECT COUNT(*) FROM departments2;
```

Salida de Script x Resultado de la Consulta x

Tarea terminada en 0,07 segundos

Table DEPARTMENTS2 creado.

Ilustración 16. Creación de departments2.

Salida de Script x Resultado de la Consulta x

Todas las Filas Recuperadas: 1 en 0,006 segundos

COUNT(*)
0

Ilustración 17. Resultado por ejecución de creación de departments2.

INSERCIÓN DE TRES TUPLAS EN DEPARTMENTS2

```
INSERT INTO departments2 (department_id, department_name, manager_id, location_id)
VALUES (300, 'Depto Ray', 100, 1700);

INSERT INTO departments2 (department_id, department_name, manager_id, location_id)
VALUES (301, 'Depto Joger', 101, 1800);

INSERT INTO departments2 (department_id, department_name, manager_id, location_id)
VALUES (302, 'Depto Prueba', NULL, 1900);
```

Ilustración 18. Script 3 tuplas.

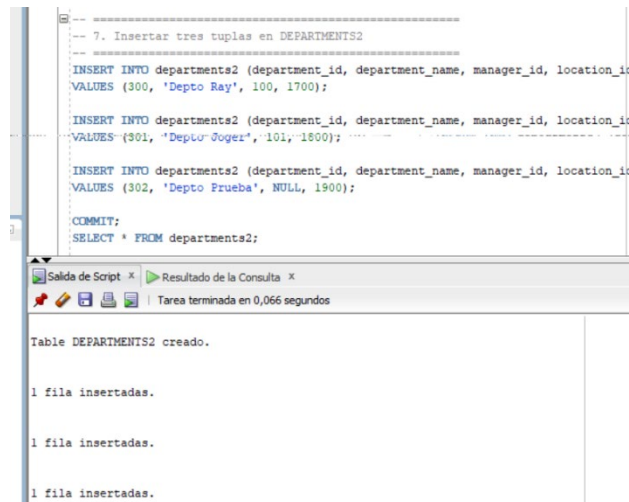


Ilustración 19. Adición realizada con éxito.

PERSISTENCIA DE DATOS AL CERRAR SESIÓN

Se cerró la sesión en SQL Developer y se volvió a conectar. Luego se consultó DEPARTMENTS2:

```
COMMIT;
SELECT * FROM departments2;
```

Ilustración 20. Script para persistencia de datos al cerrar sesión.

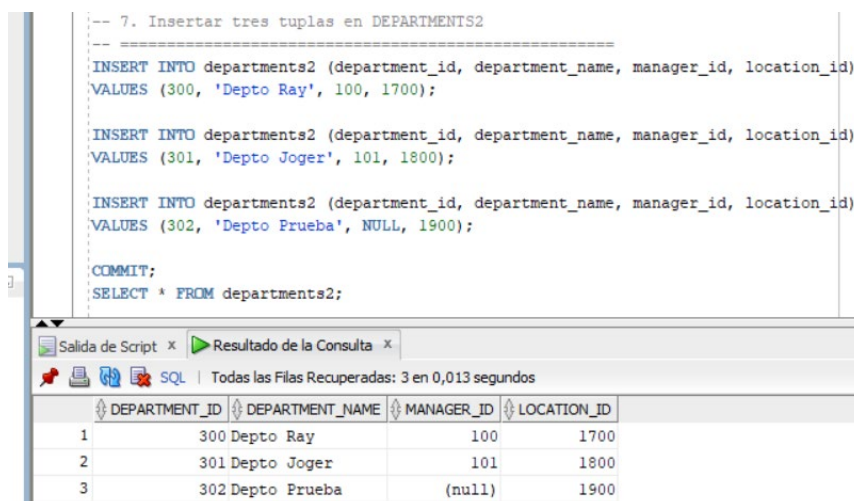


Ilustración 21. Ejecución de persistencia de datos al cerrar sesión.

BLOQUE ANÓNIMO CON TRANSACCIÓN

Se creó un bloque PL/SQL que demuestra una transacción explícita:

```

DECLARE
    v_count NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('=== INICIO DE TRANSACCION ===');

    INSERT INTO departments2 VALUES (303, 'Depto Transaccion', 102, 2000);
    UPDATE departments2 SET location_id = 2500 WHERE department_name = 'Depto Ray';

    SELECT COUNT(*) INTO v_count FROM departments2;
    DBMS_OUTPUT.PUT_LINE('Registros en departments2: ' || v_count);

    DBMS_OUTPUT.PUT_LINE('=== FIN DE TRANSACCION ===');
    COMMIT;
END;
```

Ilustración 22. Script bloque anónimo con transacción.

```

-- =====
-- 8. Bloque anónimo con transacción
-- =====

DECLARE
    v_count NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('=== INICIO DE TRANSACCION ===');

    INSERT INTO departments2 VALUES (303, 'Depto Transaccion', 102, 2000);
    UPDATE departments2 SET location_id = 2500 WHERE department_name = 'Depto Ray';

    SELECT COUNT(*) INTO v_count FROM departments2;
    DBMS_OUTPUT.PUT_LINE('Registros en departments2: ' || v_count);

    DBMS_OUTPUT.PUT_LINE('=== FIN DE TRANSACCION ===');
    COMMIT;
END;
/

SELECT * FROM departments2;
```

Salida de Script x Resultado de la Consulta x

Tarea terminada en 0,052 segundos

Confirmación terminada.
 === INICIO DE TRANSACCION ===
 Registros en departments2: 4
 === FIN DE TRANSACCION ===

Procedimiento PL/SQL terminado correctamente.

Ilustración 23. Ejecución de bloque anónimo con transacción.

DESHACER UNA TRANSACCIÓN CON ROLLBACK

Oracle permite deshacer cambios no confirmados mediante ROLLBACK:

```

INSERT INTO departments2 VALUES (304, 'Depto a Eliminar', 103, 2100);
SELECT * FROM departments2 WHERE department_id = 304;
ROLLBACK;
SELECT * FROM departments2 WHERE department_id = 304;
```

Ilustración 24. Script para deshacer una transacción con roolback.

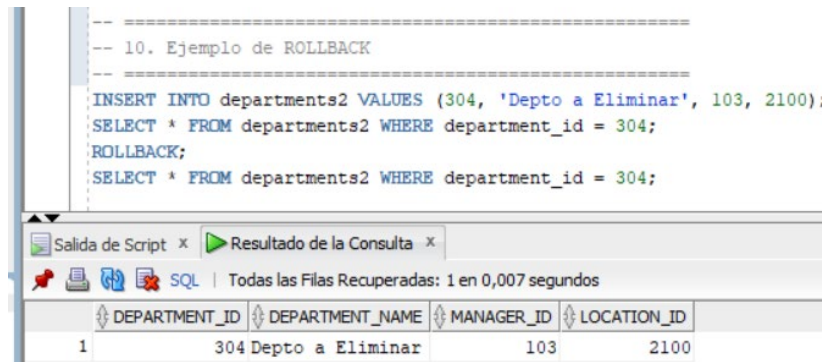


Ilustración 25. Antes de deshacer el cambio.

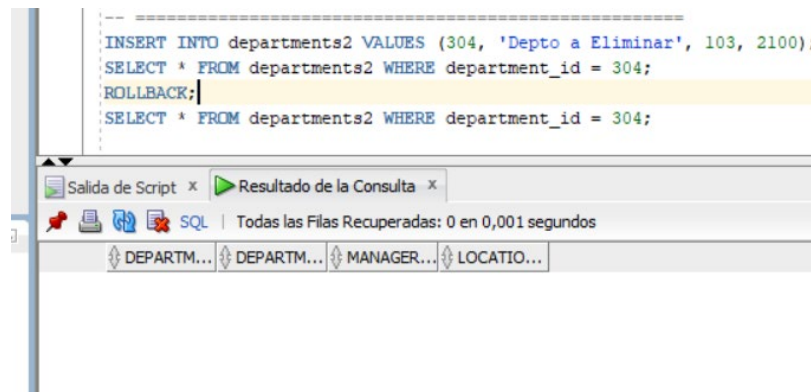


Ilustración 26. Después de deshacer el cambio.

ROLLBACK solo afecta cambios desde el último COMMIT. Los cambios confirmados no se pueden deshacer con ROLLBACK.

ARCHIVOS REDO EN ORACLE

Los archivos Redo (redo logs) son archivos donde Oracle registra secuencialmente todas las operaciones de cambio realizadas sobre la base de datos (inserciones, actualizaciones, eliminaciones). Su propósito principal es permitir la recuperación ante fallos: si la base de datos se cae inesperadamente, Oracle puede reaplicar los cambios registrados en los Redo logs para restaurar la consistencia.

Modos de funcionamiento:

- **NOARCHIVELOG:** Los Redo logs se reutilizan en círculo. No se guarda un historial completo. Solo permite recuperación hasta el último backup completo. Es el modo por defecto en muchas instalaciones de desarrollo.
- **ARCHIVELOG:** Oracle archiva los Redo logs antes de reutilizarlos, guardando un historial completo de todos los cambios. Permite recuperación punto-en-el-tiempo. Es obligatorio en entornos de producción.

Consulta teórica: Para verificar el modo actual, un usuario con privilegios de administrador ejecutaría:



Ilustración 27. Script para redo en oracle.

En una instalación típica de Oracle 21c XE con el esquema HR, el resultado esperado es *NOARCHIVELOG*, ya que es la configuración por defecto para bases de datos de desarrollo.

REPOSITORIO DE GITHUB

El repositorio contiene el script SQL completo:

https://github.com/JogerCastillo/joger_castillo_ray_gonzalez_indices_restricciones.git

CONCLUSIONES

Este trabajo permitió comprender en la práctica:

- Los índices en Oracle están asociados a constraints PRIMARY KEY y UNIQUE.
- Las restricciones pueden desactivarse temporalmente con DISABLE y reactivarse con ENABLE.
- Oracle no permite activar una restricción si los datos existentes la violan.
- COMMIT hace permanentes los cambios; ROLLBACK los deshace si no fueron confirmados.
- Los archivos Redo son fundamentales para la recuperación ante fallos.